



ALNS on Integrated Healthcare Timetabling

Bello Renato, Martin Emanuele, Terenziani Andrea

May 4, 2025



**Politecnico
di Torino**



Table of Contents

1 Data Structures

- ▶ Data Structures
- ▶ Hard and Soft Constraints
- ▶ Adaptive Large Neighbours Search implementation
- ▶ Results



Data Structures

1 Data Structures

The folder `instances` contains all the main classes used to model the problem:

- **Occupant:** Represents individuals who are already hospitalized from day 0. These are not surgical patients, but require supervision by nurses during their stay.
- **Patient:** Similar to *Occupant*, but with additional attributes like *compatible rooms*, *surgeon*, *mandatory surgery release day* *surgery due day*
- **Nurse:** Stores information about each nurse in the hospital.
- **Surgeon:** Stores information about each surgeon in the hospital.
- **Rooms:** Defined in `hospital.py`, this class manages room-related data. It includes the following methods: `add_occupant`, `add_patient`, `remove_patient`.



Data Structures

1 Data Structures

- **Theaters:** Defined in `hospital.py`, this class manages data regarding the operating theaters.
- **Time:** Also in `hospital.py`, this class stores scheduling information, including the total number of days and shifts to be considered.
- **Solution:** Represents a candidate solution to the problem. It consists of two main components: *nurse schedule*, *patient schedule*.
- **Problem:** This class takes a `Solution` object and verifies whether it satisfies all hard constraints. It also evaluates the soft constraints, interpreting them as part of a cost function to be minimized.



Table of Contents

2 Hard and Soft Constraints

► Data Structures

► **Hard and Soft Constraints**

► Adaptive Large Neighbours Search implementation

► Results



Hard Constraints 1

2 Hard and Soft Constraints

- **H1:** No gender mixing is allowed within a room; all patients in a room must have the same gender.
- **H2:** Each patient must be assigned to one of their compatible rooms and must remain in the same room for the entire duration of their stay.
- **H3:** Each surgeon must not exceed their maximum daily working time for performing surgeries.
- **H4:** Each operating theater must not exceed its maximum daily operational time.



Hard Constraints 2

2 Hard and Soft Constraints

- **H5:** All mandatory patients must be admitted to the hospital.
- **H6:** Each patient must be admitted within their allowed admission window. For mandatory patients, this means between the release day and the due day. Non-mandatory patients, if admitted, must be admitted on or after their release day.
- **H7:** Room occupancy must not exceed the maximum capacity of the room.
- **H8:** Every non-empty room must be supervised by exactly one nurse.



Soft Constraints 1

2 Hard and Soft Constraints

- **S1:** Minimize the maximum age difference among patients within the same room.
- **S2:** Minimize the number of nurses assigned to rooms where their skill level is insufficient (the skill level of a room is computed as the maximum skill level required by the patients/occupants in that room).
- **S3:** Minimize the number of nurses that providers care to each given patient or occupant.
- **S4:** Minimize the discrepancy between the required workload in each room and the capacity of the assigned nurse for every day and shift.



Soft Constraints 2

2 Hard and Soft Constraints

- **S5:** Minimize the number of operating theaters that are open each day.
- **S6:** Minimize the number of different operating theaters assigned to the same surgeon for each day.
- **S7:** Minimize the length of hospital stays, i.e., the number of days between a patient's admission and discharge.
- **S8:** Minimize the number of non-mandatory patients who are not admitted to the hospital.



Table of Contents

3 Adaptive Large Neighbours Search implementation

- ▶ Data Structures
- ▶ Hard and Soft Constraints
- ▶ Adaptive Large Neighbours Search implementation
- ▶ Results



Algorithm

3 Adaptive Large Neighbours Search implementation

Algorithm 1 Adaptive Large Neighborhood Search (ALNS)

```
1: Initialize solution  $s \leftarrow s_0$ 
2: Set best solution  $s^* \leftarrow s$ 
3: Initialize weights for destroy and repair operators
4: for each iteration do
5:   Select a destroy operator  $d$  based on weights
6:   Select a repair operator  $r$  based on weights
7:   Generate new solution  $s' \leftarrow r(d(s))$ 
8:   if  $accept(s, s')$  then
9:      $s \leftarrow s'$ 
10:    if  $f(s') < f(s^*)$  then
11:       $s^* \leftarrow s'$ 
12:    end if
13:  end if
14:  Update operator weights
15: end for
16: return  $s^*$ 
```



Destroy Operators 1

3 Adaptive Large Neighbours Search implementation

Here presented all the *destroy* operations:

- **A:** Select randomly a day, shift and room and remove all the other nurses from the schedule in the solution that don't work during that day and shift in that room selected.
- **B:** Select randomly a day, shift and room and remove all the other nurses from the schedule in the solution that work during that day and shift in that room selected.
- **C:** Select randomly some days, shifts and rooms and remove all the nurses from the schedule in the solution that work during all the combinations of the days, shifts and rooms selected.
- **D:** Select randomly a patient from the solution schedule and remove all the other patient from schedule.
- **E:** Select randomly a patient from the solution schedule and remove that patient.
- **F:** Select randomly a group of patients and remove them.



Destroy operators 2

3 Adaptive Large Neighbours Search implementation

- **G**: Is a mix of destroy operations *A* and *D*.
- **H**: Is a mix of destroy operations *A* and *E*.
- **I**: Is a mix of destroy operations *A* and *F*.
- **L**: Is a mix of destroy operations *B* and *D*.
- **M**: Is a mix of destroy operations *B* and *E*.
- **N**: Is a mix of destroy operations *B* and *F*.
- **O**: Is a mix of destroy operations *C* and *D*.
- **P**: Is a mix of destroy operations *C* and *E*.
- **Q**: Is a mix of destroy operations *C* and *F*.



Repair operators 1

3 Adaptive Large Neighbours Search implementation

All the *repair* operators are divided into two part: repairing the schedule of the patients, repairing the schedule of the nurses.

The first part is common to all the *repair* operators and it's based on schedule of all the mandatory patients that where destroyed, for aim that the program removes not mandatory patients from the hospital until it finds a place for all the destroyed mandatory patients, after that it tries to find place for the not mandatory patients that still not in the hospital.



Repair operator 2

3 Adaptive Large Neighbours Search implementation

The second part of the *repair* operations we found 4 different methods for repairing the schedule of the nurses:

- **A:** With the same logic used for finding the initial solution, we cycle over the days and shifts and assigned a nurse to each room that is nurses empty (because the destroy operator might have destroyed some nurses schedule).
- **B:** The logic is the same of the case A, but we try to assign a nurse to a room that minimize the difference between the skill level required in the room and the skill level of the nurse.
- **C:** The logic is the same of the case A, but we assign a nurse to a room trying to minimize the exceeded workload of the nurse.
- **D:** It's a mix of the logic B and C.



Table of Contents

4 Results

- ▶ Data Structures
- ▶ Hard and Soft Constraints
- ▶ Adaptive Large Neighbours Search implementation
- ▶ Results



Results over the tests

4 Results

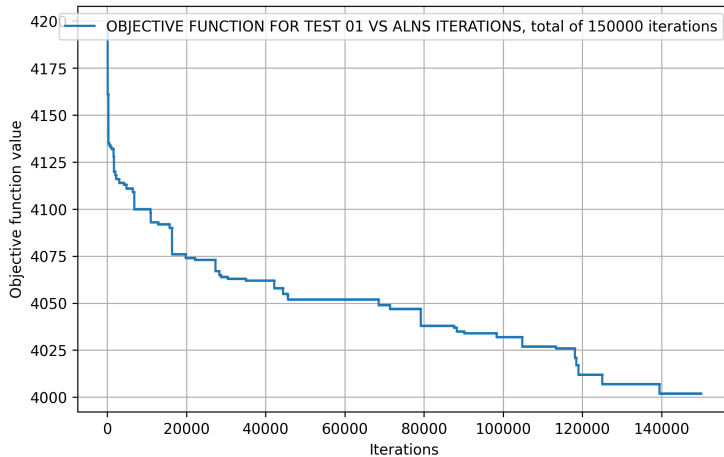
Test	Iterations	Op. Value	ALNS Value	ρ	$[\delta_1, \delta_2, \delta_3, \delta_4]$	γ	Total time	Destroy / Repair Time
1	150000	3177	4002	0.001	[8, 5, 1.5, 0.5]	0.9	7705.14	3952.54 / 3752.61
2	50000	1583	2375	0.9	[8.5, 8, 4, 0.5]	0.3	705.77	358.60 / 347.17
3	10000	10184	11082	0.1	[8, 5, 1.5, 0.5]	1.5	507.44	260.36 / 247.07
4	250000	2332	3103	0.1	[8, 5, 1.5, 0.5]	0.9	22526.26	11667.99 / 10858.27
5	1000	15713	16922	0.3	[8, 3, 2, 0.5]	0.5	23.05	11.59 / 11.46
6	40000	18558	24905	0.001	[8, 5, 1.5, 0.5]	0.9	2625.35	1307.61 / 1317.74
7	30000	17048	20243	0.001	[8, 5, 1.5, 0.5]	0.9	4262.67	2200.35 / 2062.32
8	30000	24947	26851	0.001	[8, 5, 1.5, 0.5]	0.9	5175.67	2665.95 / 2509.72
9	35000	20492	25600	0.001	[8, 5, 1.5, 0.5]	1.5	9513.41	4971.15 / 4542.25
10	10000	49444	84594	0.001	[8, 5, 1.5, 0]	0.9	7937.11	3895.94 / 4041.18

Table: Summary of test results, tuning parameters, and execution times



Plots pt.1

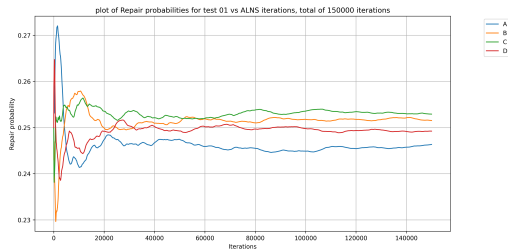
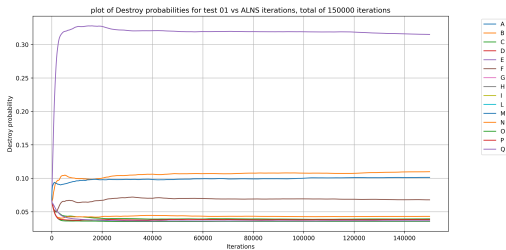
4 Results





Plots pt.2

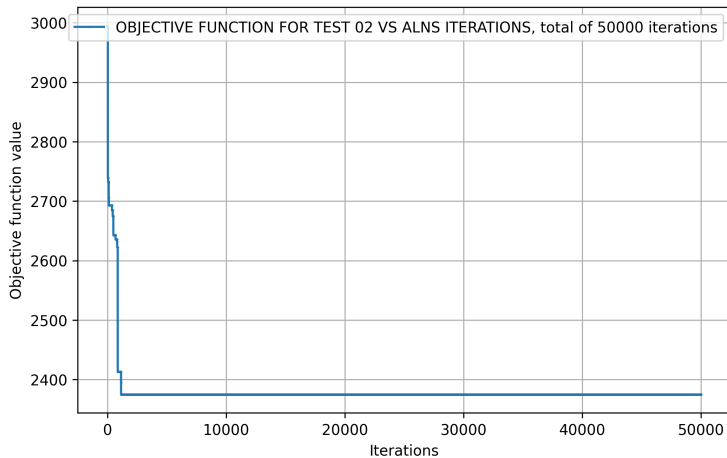
4 Results





Plots pt.3

4 Results





Plots pt.4

4 Results

